

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 1

GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	PRUEBAS DE SOFTWARE				
TÍTULO DE LA PRÁCTICA:	Pruebas de Rendimiento y Seguridad				
NÚMERO DE PRÁCTICA:	9	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
TIPO DE PRÁCTICA:	INDIVIDUAL				
	GRUPAL	X	MÁXIMO DE ESTUDIANTES	4	
FECHA INICIO:	06-07-2026	FECHA FIN:	10-07-2026	DURACIÓN:	100 min
RECURSOS A UTILIZAR: • Computadora personal • Python 3.12 o superior • Visual Studio Code (Python, Flask, Requests) • Apache JMeter 5.6+ • Herramienta K6					
DOCENTE(s): • Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto					

OBJETIVOS/TEMAS Y COMPETENCIAS
OBJETIVOS: • Diseñar escenarios de pruebas de rendimiento utilizando Apache JMeter. • Automatizar pruebas de carga mediante scripts con k6. • Implementar una API REST en Python utilizando Flask para evaluar su desempeño. • Analizar métricas de rendimiento obtenidas durante la ejecución de pruebas concurrentes. • Ejecutar pruebas básicas de seguridad sobre servicios REST utilizando Python. • Interpretar indicadores de desempeño para identificar posibles cuellos de botella en aplicaciones web.
TEMAS: • Pruebas No Funcionales. • Pruebas de Rendimiento.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 2

	● Pruebas de Carga. ● Pruebas de Estrés. ● Pruebas Spike. ● Pruebas de Resistencia. ● Métricas de Rendimiento. ● Automatización con Python. ● Pruebas básicas de Seguridad para APIs REST.
COMPETENCIAS	C.m. Construye responsablemente soluciones siguiendo un proceso adecuado llevando a cabo las pruebas ajustadas a los recursos disponibles del cliente

CONTENIDO DE LA GUÍA	
I. MARCO CONCEPTUAL 1. Pruebas de Rendimiento Las pruebas de rendimiento constituyen una categoría de pruebas no funcionales cuyo propósito es evaluar el comportamiento de un sistema bajo diferentes niveles de carga de trabajo. A diferencia de las pruebas funcionales, cuyo objetivo consiste en verificar que el software produzca resultados correctos, las pruebas de rendimiento analizan la velocidad de respuesta, estabilidad, utilización de recursos y capacidad de procesamiento del sistema. Actualmente estas pruebas forman parte esencial de los procesos DevOps y Continuous Testing, permitiendo detectar problemas de escalabilidad antes de que las aplicaciones sean desplegadas en ambientes productivos. Entre los principales objetivos de las pruebas de rendimiento destacan: ● Determinar el tiempo promedio de respuesta. ● Medir la capacidad máxima de usuarios concurrentes. ● Identificar cuellos de botella. ● Evaluar el consumo de CPU, memoria y ancho de banda. ● Validar la estabilidad del sistema durante periodos prolongados de ejecución. 2. Tipos de Pruebas de Rendimiento Dependiendo del objetivo de evaluación, las pruebas de rendimiento pueden clasificarse en diferentes categorías. Load Testing (Prueba de Carga) Evalúa el comportamiento del sistema cuando trabaja bajo una cantidad esperada de usuarios concurrentes.	

Ejemplo:

- 100 usuarios consultando un catálogo de productos simultáneamente.

Su finalidad consiste en verificar que el tiempo de respuesta permanezca dentro de los límites aceptables definidos por los requerimientos del negocio.

Stress Testing (Prueba de Estrés)

Consiste en incrementar progresivamente la carga hasta provocar el fallo del sistema.

Permite determinar:

- Punto máximo de capacidad.
- Recuperación ante fallos.
- Comportamiento bajo sobrecarga.

Estas pruebas permiten conocer el límite operativo de una aplicación antes de presentar degradaciones críticas en el servicio.

Spike Testing

Analiza el comportamiento del sistema cuando ocurre un incremento repentino o súbitos en el número de usuarios (tráfico).

Ejemplo:

Un sistema universitario recibe normalmente 300 usuarios por minuto, pero durante la matrícula alcanza repentinamente 5 000 usuarios simultáneos.

La prueba verifica si el sistema logra absorber dicho incremento y recuperarse posteriormente.

Endurance Testing

También denominada prueba de resistencia.

Consiste en mantener una carga constante durante varias horas para detectar:

- Fugas de memoria.
- Saturación de conexiones.
- Consumo progresivo de recursos.
- Degradación del rendimiento.

Este tipo de prueba resulta especialmente importante en servidores web y aplicaciones empresariales que permanecen operativas las 24 horas del día.

3. Métricas de Rendimiento

Durante la ejecución de una prueba de rendimiento no basta con verificar si el sistema responde correctamente; también es necesario medir indicadores que permitan evaluar objetivamente su

desempeño. Estas métricas proporcionan información sobre la capacidad del sistema para soportar diferentes niveles de carga y constituyen la base para la toma de decisiones orientadas a la optimización.

Las principales métricas utilizadas en la industria son:

Tiempo de Respuesta (Response Time)

Corresponde al tiempo total que transcurre desde que un usuario envía una solicitud hasta que recibe la respuesta del servidor.

Generalmente se expresa en milisegundos (ms).

Ejemplo:

- Excelente: menor a 500 ms
- Bueno: entre 500 ms y 1 segundo
- Aceptable: entre 1 y 2 segundos
- Deficiente: mayor a 2 segundos

Throughput

Representa la cantidad de solicitudes procesadas por el servidor durante un intervalo de tiempo.

Puede expresarse como:

- Requests por segundo (RPS)
- Transactions por segundo (TPS)

Mientras mayor sea el Throughput, mayor será la capacidad del sistema para atender usuarios concurrentes.

Latencia

Es el tiempo que tarda una solicitud en comenzar a ser atendida por el servidor.

- Una alta latencia suele indicar:
 - Congestión de la red.
 - Saturación del servidor.
 - Problemas de infraestructura.
- Error Rate

Representa el porcentaje de solicitudes que no fueron procesadas correctamente.

Incluye errores como:

- HTTP 400
- HTTP 401
- HTTP 403
- HTTP 404

- HTTP 500
- Timeout
- Conexión rechazada

En aplicaciones críticas el porcentaje de errores debería ser cercano al 0%.

Percentiles

Los percentiles permiten conocer el comportamiento de la mayoría de las solicitudes.

Por ejemplo:

- Percentil 90
Indica que el 90% de las peticiones respondió antes del tiempo indicado.
- Percentil 95
Es una de las métricas más utilizadas en la industria para medir la experiencia del usuario.
- Percentil 99
Permite identificar solicitudes extremadamente lentas.

Los percentiles ofrecen una visión más representativa que el simple promedio.

Consumo de Recursos

Durante las pruebas también se monitorean los recursos utilizados por el servidor:

- Uso de CPU.
- Uso de memoria RAM.
- Espacio en disco.
- Número de conexiones activas.
- Consumo de ancho de banda.

Estas métricas ayudan a identificar cuellos de botella y posibles problemas de escalabilidad.

4. Automatización de Pruebas con Python

Python se ha convertido en uno de los lenguajes más utilizados para la automatización de pruebas debido a su sintaxis sencilla, gran cantidad de bibliotecas y facilidad para integrarse con herramientas de integración continua.

Entre sus principales ventajas destacan:

- Código fácil de leer y mantener.
- Amplia comunidad de desarrollo.
- Gran cantidad de librerías para pruebas.
- Integración con APIs REST.
- Compatibilidad con herramientas DevOps.

En esta práctica se utilizarán principalmente las siguientes bibliotecas:

Flask

Framework ligero utilizado para desarrollar servicios web REST.

Permite construir rápidamente aplicaciones web y APIs necesarias para ejecutar pruebas de rendimiento.

Requests

Biblioteca utilizada para realizar solicitudes HTTP desde Python.

Permite automatizar pruebas como:

- GET
- POST
- PUT
- DELETE
- Envío de parámetros
- Autenticación
- Verificación de respuestas

pytest (Opcional)

Framework para automatizar pruebas unitarias y de integración.

Puede utilizarse para verificar automáticamente que la API continúe funcionando correctamente antes de ejecutar pruebas de rendimiento.

5. Apache JMeter

Apache JMeter es una herramienta Open Source desarrollada por la Apache Software Foundation para realizar pruebas de carga, rendimiento y estrés sobre aplicaciones web, servicios REST, bases de datos y otros protocolos de comunicación.

Su funcionamiento se basa en la simulación de múltiples usuarios virtuales que ejecutan solicitudes simultáneas hacia una aplicación.

Entre sus principales características se encuentran:

- Interfaz gráfica intuitiva.
- Generación de miles de usuarios virtuales.
- Reportes automáticos.
- Gráficos estadísticos.
- Exportación de resultados.
- Integración con Jenkins y GitHub Actions.

Los principales componentes utilizados en esta práctica serán:

Test Plan

Documento principal donde se define toda la prueba.

Thread Group

Representa el conjunto de usuarios virtuales.

Permite configurar:

- Número de usuarios.
- Ramp-Up Period.
- Número de iteraciones.

HTTP Request

Define las peticiones que realizarán los usuarios.

Puede configurarse para utilizar los métodos:

- GET
- POST
- PUT
- DELETE

Listeners

Permiten visualizar los resultados obtenidos durante la ejecución.

Los más utilizados son:

- View Results Tree
- Summary Report
- Aggregate Report
- Graph Results
- Response Time Graph

6. k6

k6 es una herramienta moderna para pruebas de rendimiento desarrollada por Grafana Labs.

A diferencia de JMeter, las pruebas se escriben mediante código JavaScript, lo que facilita su integración con sistemas de control de versiones y procesos DevOps.

Sus principales ventajas son:

- Alto rendimiento.
- Bajo consumo de memoria.
- Automatización mediante scripts.
- Integración con Docker.
- Integración con GitHub Actions.
- Integración con Grafana para monitoreo en tiempo real.

Un script básico de k6 está compuesto por tres elementos:

- Importación de módulos.
- Configuración de usuarios virtuales.

- Función principal que ejecuta las solicitudes HTTP.

Esta metodología permite crear escenarios de prueba altamente personalizables y reutilizables en proyectos de desarrollo continuo.

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

Ejercicio 1: Desarrollo de una API REST en Python utilizando Flask

En este ejercicio se implementará una API REST sencilla que permitirá administrar un catálogo de productos. Posteriormente, esta API será utilizada como objetivo de las pruebas de rendimiento mediante Apache JMeter y k6.

Parte 1. Configuración del entorno de desarrollo

Paso 1. Crear la carpeta del proyecto

Crear una carpeta denominada:

Laboratorio09

Abrir Visual Studio Code sobre dicha carpeta.

Paso 2. Crear un entorno virtual

Abrir una terminal y ejecutar:

```
python -m venv venv
```

Activar el entorno virtual.

Windows

```
venv\Scripts\activate
```

Linux / Mac

```
source venv/bin/activate
```

Paso 3. Instalar las librerías necesarias

```
pip install flask  
pip install requests
```


Verificar la instalación mediante:

```
pip list
```

Debe aparecer Flask y Requests entre las librerías instaladas.

Parte 2. Creación de la API

Crear un archivo denominado

```
app.py
```

Copiar el siguiente código:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

productos = [
    {"id":1,"nombre":"Laptop","precio":3500},
    {"id":2,"nombre":"Mouse","precio":90},
    {"id":3,"nombre":"Teclado","precio":180}
]

@app.route("/productos", methods=["GET"])
def listar():

    return jsonify(productos)

@app.route("/productos", methods=["POST"])
def registrar():

    datos=request.json

    nuevo={
        "id":len(productos)+1,
        "nombre":datos["nombre"],
        "precio":datos["precio"]
    }

    productos.append(nuevo)

    return jsonify(nuevo),201

@app.route("/productos/<int:id>", methods=["GET"])
def obtener(id):

    for producto in productos:
        if producto["id"]==id:
            return jsonify(producto)
    return jsonify({"mensaje":"Producto no encontrado"}),404
```

```
@app.route("/productos/<int:id>", methods=["PUT"])
def actualizar(id):

    datos=request.json

    for producto in productos:
        if producto["id"]==id:
            producto["nombre"]=datos["nombre"]
            producto["precio"]=datos["precio"]
            return jsonify(producto)

    return jsonify({"mensaje":"Producto no encontrado"}),404

@app.route("/productos/<int:id>", methods=["DELETE"])
def eliminar(id):

    global productos
    productos=[p for p in productos if p["id"]!=id]
    return jsonify({"mensaje":"Producto eliminado"})

if __name__=="__main__":
    app.run(debug=True)
```

Parte 3. Ejecutar el servidor

En la terminal ejecutar:

```
python app.py
```

El servidor iniciará en:

```
http://127.0.0.1:5000 o http://localhost:5000
```

Deberá mostrarse un mensaje similar al siguiente:

```
Running on http://127.0.0.1:5000
```

No cierre esta ventana durante el laboratorio.

Parte 4. Verificación de la API

Abrir el navegador e ingresar:

```
http://localhost:5000/productos
```

Debe obtener una respuesta similar a:

```
[
  {
    "id":1,
    "nombre":"Laptop",
    "precio":3500
  },
  {
    "id":2,
    "nombre":"Mouse",
```

```
"precio":90
},
{
  "id":3,
  "nombre":"Teclado",
  "precio":180
}
]
```

La correcta visualización de esta información indica que la API está lista para ser sometida a pruebas de rendimiento.

Parte 5. Verificación mediante Python Requests

Crear un nuevo archivo llamado

cliente.py

Escribir el siguiente código:

```
import requests

url="http://localhost:5000/productos"

respuesta=requests.get(url)

print("Código:",respuesta.status_code)

print("Respuesta:")

print(respuesta.json())
```

Ejecutar:

```
python cliente.py
```

La salida esperada será similar a:

Código: 200

Respuesta:

```
[
  {'id':1,'nombre':'Laptop','precio':3500},
  {'id':2,'nombre':'Mouse','precio':90},
  {'id':3,'nombre':'Teclado','precio':180}
]
```

Con esta prueba se verifica que el servicio REST responde correctamente antes de iniciar las pruebas de carga. Esta validación inicial permite asegurar que los resultados obtenidos posteriormente con Apache

JMeter y k6 correspondan al comportamiento del sistema bajo carga y no a errores de implementación de la API.

Parte 6. Pruebas de Rendimiento utilizando Apache JMeter

Una vez verificado el correcto funcionamiento de la API desarrollada en Flask, se procederá a evaluar su desempeño utilizando Apache JMeter.

Descargue JMeter 5.6.3: https://jmeter.apache.org/download_jmeter.cgi

El objetivo es simular múltiples usuarios concurrentes realizando solicitudes HTTP sobre el servicio REST para medir su capacidad de respuesta y estabilidad.

Paso 1. Iniciar Apache JMeter

Requiere Java 8+

Abrir Apache JMeter (busca el archivo jmeter.bat ubicada dentro de la carpeta /bin/)

Al iniciar la aplicación aparecerá automáticamente un **Test Plan** vacío.

Paso 2. Crear un Thread Group

Seleccionar:

Test Plan- Add-Threads (Users)-Thread Group

Configurar los siguientes parámetros:

Parámetro	Valor
Number of Threads	20
Ramp-Up Period	10 segundos
Loop Count	5

Esto simulará 20 usuarios concurrentes que accederán progresivamente al servidor.

Paso 3. Agregar un HTTP Request

Seleccionar:

Thread Group-Add-Sampler-HTTP Request

Configurar:

Campo	Valor
Protocol	http
Server Name	localhost
Port	5000
Method	GET

Path

/productos

Paso 4. Agregar Listeners

Añadir los siguientes componentes:

- Summary Report
- Aggregate Report
- View Results Tree
- Graph Results

Estos reportes permitirán analizar el comportamiento del servidor durante la prueba.

Paso 5. Ejecutar la prueba

Presionar el botón **Start**.

Esperar a que todos los usuarios finalicen la ejecución.

Registrar los siguientes indicadores:

- Tiempo promedio de respuesta.
- Tiempo mínimo.
- Tiempo máximo.
- Throughput.
- Desviación estándar.
- Porcentaje de errores.

Paso 6. Incrementar la carga

Modificar el Thread Group y ejecutar nuevamente las pruebas.

Escenario 1

Usuarios	Ramp-Up
20	10 segundos

Escenario 2

Usuarios	Ramp-Up
50	20 segundos

Escenario 3

Usuarios	Ramp-Up
100	30 segundos

Registrar los resultados

Completar la siguiente tabla.

Usuarios	Tiempo Promedio (ms)	Throughput	Error %
20			
50			
100			

Posteriormente responder:

- ¿Cómo varía el tiempo de respuesta conforme aumenta el número de usuarios?
- ¿Existe degradación del rendimiento?
- ¿Qué recurso del servidor podría estar limitando el desempeño?

Parte 7. Pruebas de Rendimiento utilizando K6

K6 permite automatizar pruebas de carga mediante scripts escritos en JavaScript.

Paso 1. Instalación

```
winget install k6 o choco install k6
```

Verificar la instalación ejecutando:

```
k6 --version
```

Paso 2. Crear el script

Crear un archivo llamado

prueba.js

Copiar el siguiente código.

```
import http from 'k6/http';
import { sleep } from 'k6';
export const options = {
  vus:20,
  duration:'30s'
};
export default function(){
  http.get('http://localhost:5000/productos');
  sleep(1);
}
```

Paso 3. Ejecutar la prueba

Abrir una terminal.

Ejecutar:

```
k6 run prueba.js
```

Al finalizar se mostrará un reporte similar al siguiente:

- HTTP Requests
- HTTP Request Duration
- Iterations
- Virtual Users
- Data Received
- Data Sent
- Checks
- Failures

Registrar las métricas obtenidas.

Paso 4. Modificar el escenario

Cambiar los parámetros del script.

Escenario A

vus:20

duration:'30s'

Escenario B

vus:50

duration:'45s'

Escenario C

vus:100

duration:'60s'

Ejecutar nuevamente las pruebas.

Registrar los resultados.

Comparación entre Apache JMeter y K6

Completar la siguiente tabla comparativa.

Característica	Apache JMeter	K6
Facilidad de uso		
Curva de aprendizaje		
Automatización		
Integración CI/CD		
Consumo de memoria		
Generación de reportes		
Escalabilidad		

Análisis de resultados

Con base en las métricas obtenidas responder:

1. ¿Qué herramienta presentó menor tiempo promedio de respuesta?
2. ¿Cuál fue el Throughput máximo alcanzado?
3. ¿Qué porcentaje de errores presentó cada herramienta?
4. ¿Cuál considera más adecuada para integrar en un proceso DevOps? Justifique su respuesta.
5. ¿Qué mejoras implementaría en la API desarrollada para soportar un mayor número de usuarios concurrentes?

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Ejercicio 1. Desarrollo de una API REST para pruebas de rendimiento

Cada grupo desarrollará una API REST utilizando **Python y Flask**, seleccionando una de las siguientes temáticas o una propuesta aprobada por el docente:

- Sistema de Biblioteca.
- Sistema de Gestión de Hospital.
- Sistema de Restaurante.
- Sistema de Inventario.

- Sistema de Matrícula Universitaria.
- Sistema de Gestión de Vehículos.
- Sistema de Reserva de Hoteles.
- Sistema de Venta de Entradas.
- Sistema de Gestión de Productos.
- Sistema de Control de Alumnos.

La API deberá implementar como mínimo los siguientes servicios REST:

- GET
- POST
- PUT
- DELETE

La información podrá almacenarse en memoria o utilizar una base de datos SQLite.

Ejercicio 2. Pruebas de Rendimiento con Apache JMeter

Utilizando la API desarrollada en el ejercicio anterior, diseñe un **Plan de Pruebas** en Apache JMeter que permita evaluar el comportamiento del sistema bajo diferentes niveles de carga.

Requerimientos

Realizar tres escenarios de prueba:

Escenario 1

- 20 usuarios concurrentes
- Ramp-Up: 10 segundos
- 5 iteraciones

Escenario 2

- 50 usuarios concurrentes
- Ramp-Up: 20 segundos
- 10 iteraciones

Escenario 3

- 100 usuarios concurrentes
- Ramp-Up: 30 segundos
- 15 iteraciones

Para cada escenario registrar:

- Tiempo promedio de respuesta.
- Tiempo mínimo.
- Tiempo máximo.
- Throughput.
- Error Rate.
- Desviación estándar.

Adjuntar capturas de pantalla de:

- Test Plan.
- Thread Group.
- HTTP Request.
- Summary Report.
- Aggregate Report.
- Graph Results.

Finalmente, elaborar una tabla comparativa analizando el comportamiento del sistema conforme aumenta el número de usuarios.

Ejercicio 3. Pruebas de Rendimiento utilizando K6

Implementar un script de pruebas en **K6** para la misma API desarrollada.

El script deberá ejecutar solicitudes HTTP hacia los endpoints implementados y evaluar el comportamiento del sistema utilizando diferentes configuraciones de usuarios virtuales.

Configuración mínima

Usuarios Virtuales	Duración
20	30 segundos
50	45 segundos
100	60 segundos

Registrar para cada escenario:

- Iteraciones ejecutadas.
- Tiempo promedio de respuesta.
- Tiempo máximo.
- Throughput.
- Solicitudes exitosas.
- Solicitudes fallidas.

- Porcentaje de errores.

Comparar los resultados obtenidos con Apache JMeter e identificar las principales diferencias.

Ejercicio 4. Pruebas Básicas de Seguridad utilizando Python

Desarrollar un programa en Python utilizando la biblioteca **Requests** para ejecutar pruebas básicas de seguridad sobre la API implementada.

El programa deberá evaluar como mínimo los siguientes escenarios:

Caso 1. Validación de recursos inexistentes

Realizar solicitudes GET hacia identificadores inexistentes.

Resultado esperado:

- Código HTTP 404.

Caso 2. Validación de datos incompletos

Enviar solicitudes POST con campos obligatorios vacíos.

Resultado esperado:

- Código HTTP 400.
- Mensaje descriptivo del error.

Caso 3. Validación de tipos de datos

Enviar valores de tipo incorrecto.

Ejemplo:

```
{  
  "nombre":12345,  
  "precio":"ABC"  
}
```

El sistema deberá rechazar la solicitud.

Caso 4. Métodos HTTP no permitidos

Intentar acceder mediante un método HTTP no soportado.

Ejemplo:

PATCH /productos

Resultado esperado:

- HTTP 405.

Caso 5. Simulación de ataques por fuerza bruta (Opcional)

Si la API implementa autenticación, realizar múltiples intentos consecutivos de inicio de sesión utilizando credenciales incorrectas.

Analizar si el sistema incorpora mecanismos como:

- Bloqueo temporal.
- Limitación de intentos.
- Registro de eventos.
- Mensajes seguros.

Entregables

Cada grupo deberá presentar:

- Código fuente completo de la API en Python (Flask).
- Script de pruebas en Apache JMeter (.jmx).
- Script de pruebas en k6 (.js).
- Programa en Python para pruebas básicas de seguridad.
- Capturas de pantalla de los resultados obtenidos.
- Informe técnico con el análisis de métricas y conclusiones.

Análisis de Resultados

Con base en los resultados obtenidos, responder:

1. ¿Cuál fue el tiempo promedio de respuesta de la API en cada escenario de carga?
2. ¿Qué herramienta presentó mayor Throughput: Apache JMeter o K6?
3. ¿En qué escenario comenzaron a presentarse errores de respuesta?
4. ¿Qué componente de la infraestructura considera que representa el principal cuello de botella?
5. ¿Qué recomendaciones implementaría para mejorar el rendimiento y la seguridad de la aplicación desarrollada?

I. CUESTIONARIO

1. Explique la diferencia entre una prueba funcional y una prueba no funcional. ¿Por qué las pruebas de rendimiento forman parte de las pruebas no funcionales?
2. Describa las diferencias entre los siguientes tipos de pruebas de rendimiento:

- Load Testing
- Stress Testing
- Spike Testing
- Endurance Testing

Mencione un escenario práctico para cada una.

3. Durante una prueba de rendimiento se obtuvieron los siguientes resultados:

- Tiempo promedio de respuesta: **850 ms**
- Throughput: **180 solicitudes/segundo**
- Error Rate: **3%**
- Percentil 95: **1.8 segundos**

Interprete cada uno de estos indicadores y determine si el sistema presenta un desempeño aceptable. Justifique su respuesta.

4. Compare Apache JMeter y K6 considerando los siguientes aspectos:

- Facilidad de uso.
- Automatización.
- Escalabilidad.
- Integración con CI/CD.
- Generación de reportes.
- Consumo de recursos.

Indique en qué tipo de proyecto utilizaría cada herramienta.

5. Durante las pruebas básicas de seguridad se detectaron las siguientes situaciones:

- Acceso a recursos inexistentes.
- Envío de parámetros inválidos.
- Métodos HTTP no permitidos.
- Intentos repetitivos de autenticación.

Explique cómo deberían responder las aplicaciones modernas ante cada uno de estos escenarios y qué códigos HTTP deberían devolverse.

II. REFERENCIAS Y BIBLIOGRAFÍA RECOMENDADAS:

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLD-001	Página: 22

- [1] Myers, G., Sandler, C., & Badgett, T. *The Art of Software Testing*. 3rd Edition. John Wiley & Sons, 2012.
- [2] Spillner, A., Linz, T., & Schaefer, H. *Software Testing Foundations*. 5th Edition. Rocky Nook, 2021.
- [3] Pressman, R., & Maxim, B. *Software Engineering: A Practitioner's Approach*. 9th Edition. McGraw-Hill, 2020.
- [4] OWASP Foundation. *OWASP Web Security Testing Guide*.
- [5] OWASP Foundation. *OWASP API Security Top 10*.

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS: <i>Ejercicios propuestos / Cuestionario</i>		INSTRUMENTOS: <i>Rúbrica</i>			
Criterio	Excelente (100%)	Bueno (80%)	Suficiente (55%)	Insuficiente (30%)	No presenta (0%)
Explicación y análisis de las pruebas realizadas (30%)	Explica claramente cada escenario de pruebas, interpreta correctamente todas las métricas obtenidas y fundamenta técnicamente sus conclusiones.	Explica adecuadamente la mayoría de los escenarios y métricas, aunque algunas conclusiones presentan poca profundidad.	Describe parcialmente las pruebas realizadas; el análisis de resultados es limitado.	Presenta explicaciones incompletas y escasa interpretación de los resultados obtenidos.	No presenta explicación ni análisis de las pruebas realizadas.
Implementación de la API y ejecución de pruebas (30%)	La API funciona correctamente; las pruebas en JMeter y k6 se ejecutan sin errores y se presentan todas las evidencias solicitadas.	La API funciona correctamente; existen pequeños errores en alguna prueba o faltan evidencias menores.	La API presenta errores parciales o solo una de las herramientas ejecuta correctamente las pruebas.	La implementación es incompleta y la mayoría de las pruebas no pueden ejecutarse correctamente.	No presenta implementación ni evidencias de ejecución.
Pruebas básicas de	Implementa correctamente todos los	Implementa la mayoría de los casos de	Ejecuta parcialmente las pruebas de	Presenta pocos casos de prueba y	No presenta las pruebas de seguridad.

seguridad (20%)	casos propuestos, identifica vulnerabilidades y propone medidas de mitigación fundamentadas.	prueba con un análisis adecuado.	seguridad con análisis limitado.	sin análisis de resultados.	
Presentación del informe (20%)	Informe ordenado, completo, correctamente estructurado, con excelente redacción, tablas, capturas, conclusiones y referencias bibliográficas.	Informe bien organizado con pocos errores de redacción o formato.	Informe aceptable, aunque presenta problemas de organización o documentación.	Informe desordenado, incompleto y con numerosos errores de redacción o formato.	No presenta el informe.